과제 7 · 플랜두씨 다이어리 2 — 인증을 붙이고, 어떻게 붙였는지 설명하기

플랜두씨 다이어리 2

검사 증거 자료 (남길 것)

카드 1~5의 "남길 것" 항목을 실제 배포 화면 캡처와 요청·응답 기록으로 정리했습니다.

카드	남길 것	쪽
1. 무엇으로 붙일지 고르고, 이유를 적기	고른 방식·이름·버전, 고르지 않은 방법, 가입·로그인·로그아웃 화면, 로그인 없이 연 화면	2
2. 비밀번호를 어떻게 맡아 두는지 보이기	방법과 이유, 저장된 값, 같은 비밀번호 두 계정, 원문 없는 기록	6
3. 들어온 사람을 어떻게 기억하는지 보이기	세션 설명, 로그아웃 전후 응답, 만료, 비밀키 점검	8
4. 남의 자료가 안 열리는 것을 보이기	양방향 읽기·수정·삭제 거절, 다른 계정 적어 보내기, 로그인 없는 요청, 목록, 건수, 소스 위치	10
5. 설명서로 묶고, 잠근 앱으로 5일 써 보기	설명서 여섯 항목, 5일 기록과 규칙 변경, 합계·평균 대조, 내보내기·계정 삭제, 가린 제출물	14

제출 주소

결과물	https://plandosee-diary-2.vercel.app (첫 화면 = 로그인 화면)
소스	https://github.com/gaeun1210/plandosee-diary-2
인증 구현 설명서	https://github.com/gaeun1210/plandosee-diary-2/blob/main/SUBMISSION.md
이어서 쓴 과제 6	https://plandosee-diary-ten.vercel.app · 커밋 a7da53e (과제 7 저장소 이력의 조상)

검사 환경

- 요청·응답 기록: 2026년 10월 13일 19:44 (Asia/Seoul), GitHub Actions에서 배포 주소에 실제 요청을 보내 `scripts/verify-auth.mjs`로 34가지 확인 — 34가지 모두 통과. 원문: `docs/evidence/auth-check-20261013-1944.md`
- 화면 캡처: 2026년 10월 19일, GitHub Actions의 Chromium(헤드리스) + Playwright 1.56으로 배포 주소를 열어 찍음(`scripts/evidence-shots.mjs`). 시험 계정 `shot_46eb00`을 무작위 비밀번호로 만들어 쓰고 마지막에 계정 삭제까지 함.
- DB 해시·5일 실험·과제 6 자료 화면은 본인이 직접 Neon SQL Editor와 내 계정(`happyeuni`)에서 확인·캡처.
- **비밀번호·세션 값·비밀키 원문은 이 문서에 없습니다.** 세션 값은 앞 4글자만 남기고 `…(가림)`, 요청 본문의 비밀번호는 `(가림)`으로 적었습니다.

카드 1 — 무엇으로 붙일지 고르고, 이유를 적기

남길 것

- 고른 방식과 이름·버전, 그리고 고르지 않은 방법과 그 이유
- 가입·로그인·로그아웃이 되는 화면
- 로그인하지 않고 자료 화면 주소를 열었을 때 나오는 로그인 화면

1-1. 고른 방식 (C91·C92)

부분	방식	이름·버전
비밀번호 보관	라이브러리	**bcryptjs 3.0.3** (bcrypt, 비용 12)
로그인 상태 기억	직접 구현 (서버 세션)	Node.js 기본 crypto(무작위 32바이트 + SHA-256), PostgreSQL `sessions` 표
서버·DB	과제 6 그대로	Express 4.22.3, pg 8.23.1, PostgreSQL(Neon, 싱가포르), Vercel

1-2. 고르지 않은 방법과 이유 (C93)

검토한 방법	고르지 않은 이유
Supabase Auth (인증 서비스)	로그아웃해도 이미 받은 접근 토큰(JWT)이 만료(기본 1시간)될 때까지 통해서 "로그아웃 뒤 같은 값 거절"을 보이기 어렵다. 무료 요금제의 켜 둔 프로젝트 2개 제한(과제 6·BYTE BACK 사용 중)으로 새 프로젝트도 만들 수 없었다.
JWT 직접 서명	서명 비밀키를 따로 관리해야 하고, 끊으려면 결국 서버에 차단 목록이 필요해 세션과 다를 게 없다.
argon2	네이티브 모듈이라 Vercel 서버리스 빌드에서 문제가 생길 수 있다(bcryptjs는 순수 JavaScript).
Neon Auth	DB를 만들 때 켤 수 있었지만 토큰 방식이고, 설명이 두 방식으로 섞이는 것을 피하려고 껐다.

1-3. 로그인하지 않고 자료 화면 주소를 열면 (C97·C03)



로그인 없이 /?plan=1#tasks 를 연 화면 — 자료 대신 로그인 화면

Pretty-print

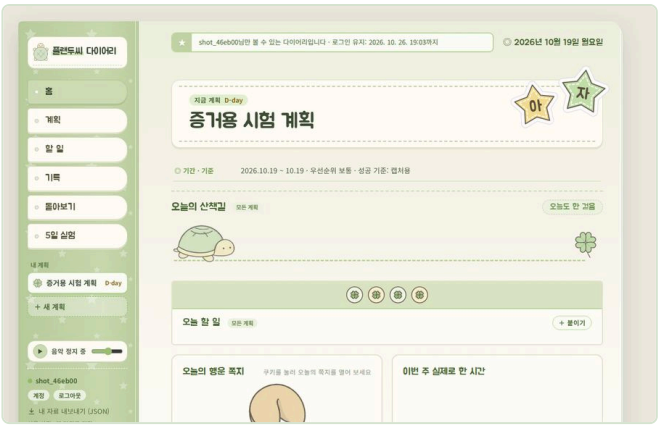
```
{"error":"로그인이 필요합니다."}
```

로그인 없이 자료 API /api/plans 를 직접 열면 {"error":"로그인이 필요합니다."}

1-4. 가입 → 로그인 상태 (C94)

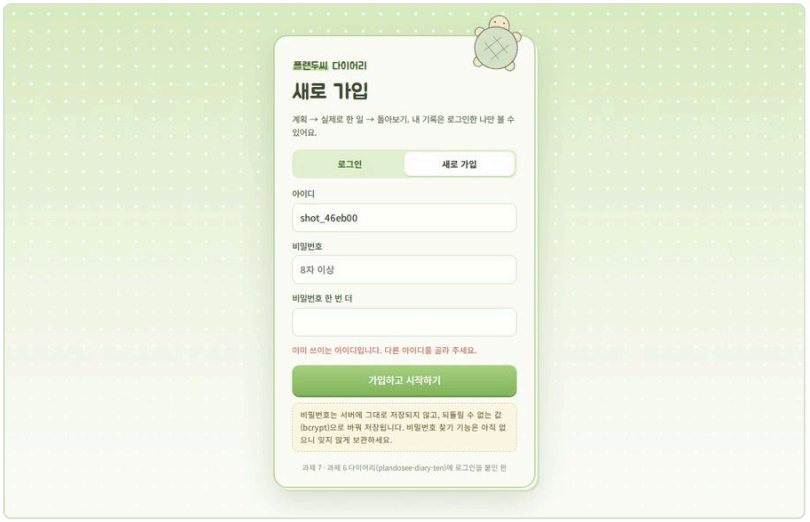


새로 가입: 비밀번호 칸은 점으로 가려짐



가입 직후: "shot_46eb00님만 볼 수 있는 다이어리입니다"

1-5. 같은 아이디로 두 번 가입되지 않음 (C98)



같은 아이디로 다시 가입 → "이미 쓰이는 아이디입니다"

같은 아이디로 다시 가입: chk_a_c4a1e7

```
POST /api/auth/signup
Content-Type: application/json
X-Requested-With: pds-diary

{"login_id":"chk_a_c4a1e7","password":"(가림)"}

→ HTTP 409
{"error":"이미 쓰이는 아이디입니다. 다른 아이디를 골라 주세요."}
```

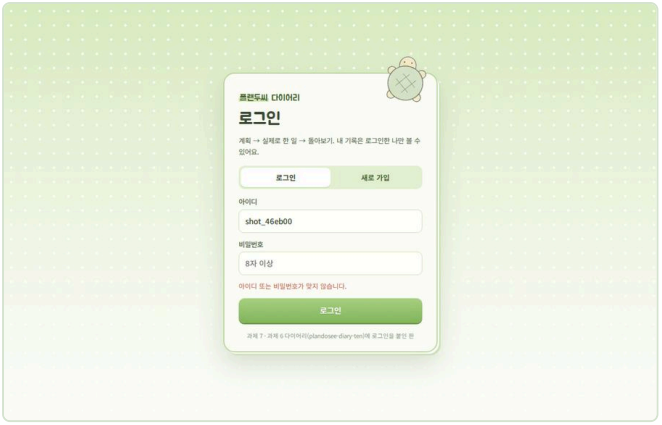
대문자로 바꿔 다시 가입: CHK_A_C4A1E7

```
POST /api/auth/signup
Content-Type: application/json
X-Requested-With: pds-diary

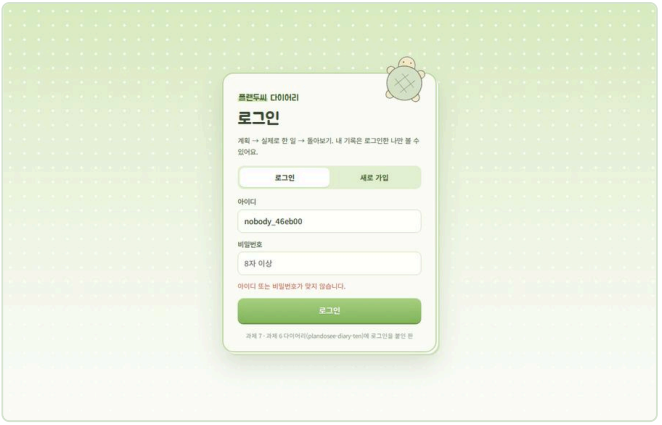
{"login_id":"CHK_A_C4A1E7","password":"(가림)"}

→ HTTP 409
{"error":"이미 쓰이는 아이디입니다. 다른 아이디를 골라 주세요."}
```

1-6. 로그인 실패 안내 문구가 같음 (C99)



있는 아이디 + 틀린 비밀번호



없는 아이디

있는 아이디 + 틀린 비밀번호

```
POST /api/auth/login
Content-Type: application/json
X-Requested-With: pds-diary

{"login_id":"chk_a_c4a1e7","password":"(가림)"}

→ HTTP 401
{"error":"아이디 또는 비밀번호가 맞지 않습니다."}
```

없는 아이디

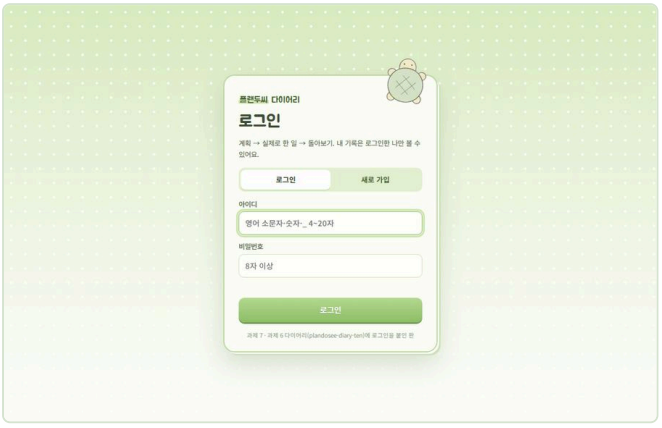
```
POST /api/auth/login
Content-Type: application/json
X-Requested-With: pds-diary

{"login_id":"nobody_c4a1e7","password":"(가림)"}

→ HTTP 401
{"error":"아이디 또는 비밀번호가 맞지 않습니다."}
```

두 경우 모두 HTTP 401, 글자까지 같은 문구.

1-7. 로그아웃 → 로그인 (C95·C96)



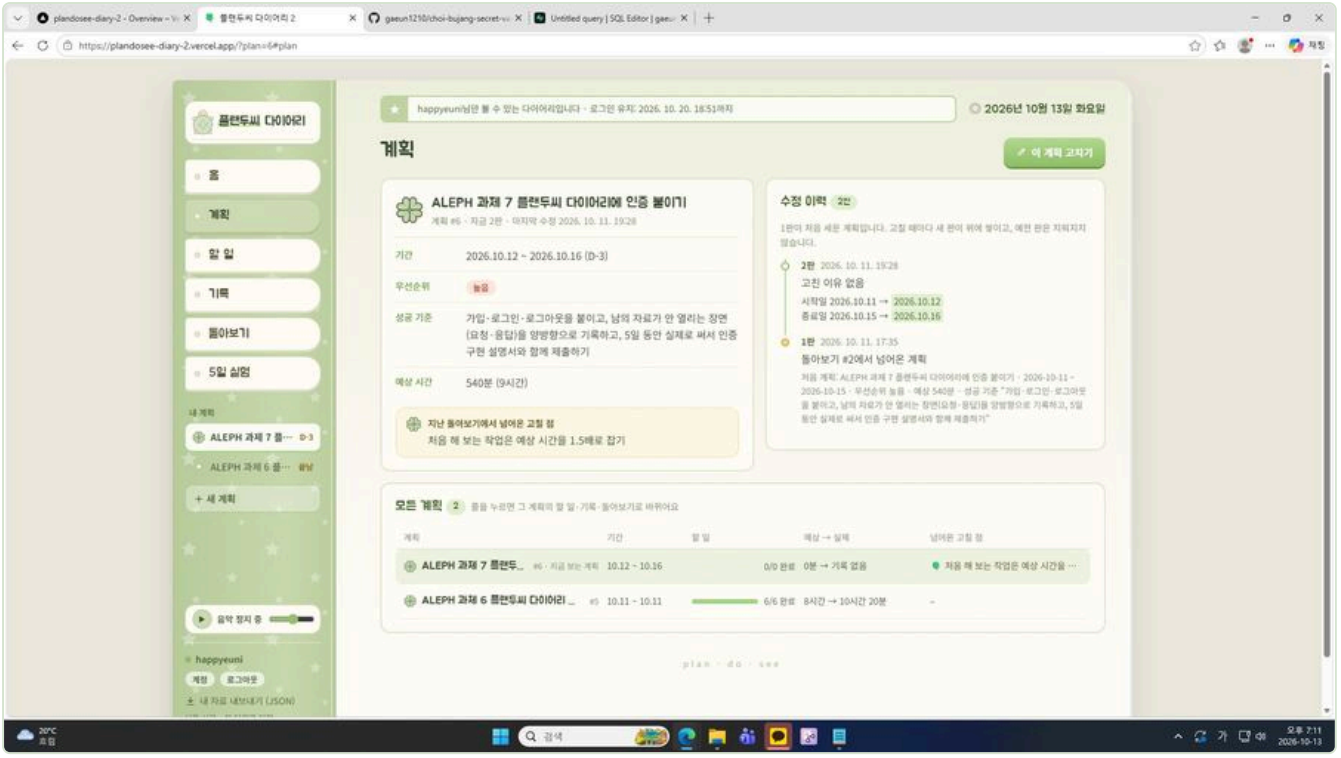
로그아웃 뒤 같은 계획 주소(/?plan=11#tasks)를 다시 열면 로그인 화면



다시 로그인하면 내 계획이 보임

1-8. 과제 6 자료를 내 계정으로 옮김 (C100)

과제 6 사이트의 "내 자료 전체 내보내기(JSON)" 파일을 과제 7의 **계정 → 과제 6 자료 가져오기**로 내 계정(`happyeuni`)에 옮겼다. 과제 6 계획(할 일 6/6 완료)과 그 돌아보기에서 넘어온 과제 7 계획이 보인다.



내 계정의 계획 화면: "모든 계획 2" — ALEPH 과제 6(#5, 6/6 완료)과 넘어온 고칠 점이 붙은 과제 7 계획(#6)

카드 2 — 비밀번호를 어떻게 맡아 두는지 보이기

남길 것

- 고른 방법의 이름과 고른 이유
- 저장된 비밀번호 값(입력한 글자가 보이지 않음)
- 같은 비밀번호로 만든 두 계정의 저장된 값이 서로 다른 결과
- 로그인 요청·응답 기록에 원문이 없는 결과

2-1. 방법과 이유 (C101·C102·C107)

bcrypt (라이브러리 bcryptjs 3.0.3, 비용 12 = 2^{12}번 반복). 계정마다 다른 소금값(salt)을 알아서 만들어 해시 안에 함께 넣어 주고, 비용을 올려 무차별 대입을 느리게 만들 수 있다. 순수 JavaScript라 Vercel 서버리스에서 따로 빌드할 것이 없다. 비밀번호를 다루는 부분은 직접 만들지 않고 이 라이브러리에 맡겼다.

2-2. DB에 저장된 모습, 같은 비밀번호 두 계정 (C103·C104)

Neon SQL Editor(`neondb`)에서 본인이 직접 조회. `chk_a_790963`과 `chk_b_790963`은 **같은 비밀번호**로 가입했다(비밀번호는 무작위로 만들어 어디에도 적지 않음).

```
select id, login_id, password_hash, length(password_hash) as len
from users where login_id like 'chk\_%' order by id;
```



Neon SQL Editor 결과 (2 rows)

login_id	password_hash
chk_a_790963	`$2b$12$aT0yHlH7zj4PoVQNUEN4MO7srz.XUZvJEnGDfocxsXvAMyHJAWJpm`
chk_b_790963	`$2b$12$2zALm/Qw0vtCVmTGH1f4ruo/AAm2OlOI52n3e1Sz0yi4YpnO/77Nq`

`$2b$` = bcrypt, `12$` = 비용, 밑줄 22글자 = 소금값, 나머지 = 해시. 입력한 글자는 보이지 않고, 소금값이 달라 같은 비밀번호라도 값 전체가 다르다.

2-3. 요청·응답 기록에 원문 없음 (C105·C106)

로그인: chk_a_c4a1e7

```
POST /api/auth/login
Content-Type: application/json
X-Requested-With: pds-diary

{"login_id":"chk_a_c4a1e7","password":"(가림)"}

→ HTTP 200
Set-Cookie: __Host-pds_sid=j1HO…(가림); Path=/; HttpOnly; SameSite=Lax; Max-Age=604799; Expires=Tue, 20 Oct 2026 10:44:56 GMT; Secure
{"user":{"id":"7","login_id":"chk_a_c4a1e7"},"session_expires_at":"2026-10-20T10:44:56.509Z"}
```

요청 본문의 비밀번호는 기록할 때 `(가림)`으로 바꾸었고, 응답에는 비밀번호도 해시도 없다. 확인 스크립트가 받은 응답 70여 개의 본문·머리글 전체에서 비밀번호 원문 등장 횟수를 셌더니 **0회**. 서버 로그(`console.error`)는 방식·경로·오류 코드만 남기고 요청 본문은 남기지 않는다. 화면은 비밀번호를 입력칸(type=password)에서만 다루고 보낸 뒤 비운다.

카드 3 — 들어온 사람을 어떻게 기억하는지 보이기

남길 것

- 무엇으로 사람을 알아보는지 적은 문장
- 같은 주소·같은 방식의 성공 응답과 로그아웃 뒤 거절 응답
- 만료 시각과 끊기는 시간
- 비밀키가 브라우저 코드·배포 파일·Git 기록에 없는 결과

3-1. 무엇으로 알아보나 (C108)

세션(토큰 아님). 로그인하면 서버가 무작위 32바이트 값을 만들어 `__Host-pds_sid` 쿠키(HttpOnly·Secure·SameSite=Lax)로 준다. DB `sessions` 표에는 그 값의 SHA-256만 저장한다. 요청이 오면 쿠키 값의 해시가 표에 있고 만료 전이면 그 사람으로 본다. 로그아웃하면 그 줄을 서버에서 지운다.

3-2. 로그아웃 전 성공, 로그아웃 뒤 같은 값으로 거절 (C109·C110)

같은 주소·같은 방식(GET)·같은 쿠키 값 `…(가림)`. 달라진 것은 그 사이의 로그아웃뿐.

성공 — 로그아웃 전

```
GET /api/auth/me
Cookie: __Host-pds_sid=j1HO…(가림)

→ HTTP 200
{"user":{"id":"7","login_id":"chk_a_c4a1e7","created_at":"2026-10-13T10:44:51.963Z"},"session_expires_at":"2026-10-20T10:44:56.509Z"}
```

거절 — 로그아웃 뒤

```
GET /api/auth/me
Cookie: __Host-pds_sid=j1HO…(가림)

→ HTTP 401
{"error":"로그인이 필요합니다."}
```

성공 — 로그아웃 전

```
GET /api/plans
Cookie: __Host-pds_sid=j1HO…(가림)

→ HTTP 200
[]
```

거절 — 로그아웃 뒤

```
GET /api/plans
Cookie: __Host-pds_sid=j1HO…(가림)

→ HTTP 401
{"error":"로그인이 필요합니다."}
```

그 사이에 보낸 로그아웃

```
POST /api/auth/logout
X-Requested-With: pds-diary
Cookie: __Host-pds_sid=j1HO…(가림)

→ HTTP 200
Set-Cookie: __Host-pds_sid=; Path=/; HttpOnly; SameSite=Lax; Max-Age=0; Expires=Thu, 01 Jan 1970 00:00:00 GMT; Secure
{"logged_out":true,"sessions_ended":1}
```

3-3. 만료 (C111)

로그인 뒤 **7일(604,800초)**이 지나면 끊긴다. 연장하지 않는다. 쿠키 `Max-Age=604799` · `Expires`와 DB `expires_at`이 같은 시각이며, 서버는 매 요청마다 `expires_at > now()`를 확인한다.

로그인 응답의 Set-Cookie와 session_expires_at

```
POST /api/auth/login
Content-Type: application/json
X-Requested-With: pds-diary

{"login_id":"chk_a_c4a1e7","password":"(가림)"}

→ HTTP 200
Set-Cookie: __Host-pds_sid=j1HO…(가림); Path=/; HttpOnly; SameSite=Lax; Max-Age=604799; Expires=Tue, 20 Oct 2026 10:44:56 GMT; Secure
{"user":{"id":"7","login_id":"chk_a_c4a1e7"},"session_expires_at":"2026-10-20T10:44:56.509Z"}
```

3-4. 비밀번호를 바꾸면 이전 값이 끊김 (C114)

성공 — 바꾸기 전

```
GET /api/auth/me
Cookie: __Host-pds_sid=pAFm…(가림)

→ HTTP 200
{"user":{"id":"11","login_id":"chk_c_c4a1e7","created_at":"2026-10-13T10:44:54.573Z"},"session_expires_at":"2026-10-20T10:44:54.577Z"}
```

거절 — 바꾼 뒤 옛 값

```
GET /api/auth/me
Cookie: __Host-pds_sid=pAFm…(가림)

→ HTTP 401
{"error":"로그인이 필요합니다."}
```

거절 — 다른 기기의 옛 값

```
GET /api/auth/me
Cookie: __Host-pds_sid=KuZa…(가림)

→ HTTP 401
{"error":"로그인이 필요합니다."}
```

성공 — 새로 받은 값

```
GET /api/auth/me
Cookie: __Host-pds_sid=0xgO…(가림)

→ HTTP 200
{"user":{"id":"11","login_id":"chk_c_c4a1e7","created_at":"2026-10-13T10:44:54.573Z"},"session_expires_at":"2026-10-20T10:45:00.024Z"}
```

3-5. 주소창·비밀키 (C112·C113·C115)

- 세션 값은 쿠키로만 오가고 주소(URL)·응답 본문에 실리지 않는다. HttpOnly라 화면 코드도 읽을 수 없다.
- 세션 값을 서명하는 **비밀키가 없는 방식**(무작위 값 + DB 대조)이라 샐 비밀키 자체가 없다. 유일한 비밀값인 DB 접속 주소(`DATABASE_URL`)는 Vercel 환경변수에만 있다.
- `git log -p --all` 전체와 배포되는 화면 파일 `public/`을 접속 주소·키 모양 검사식으로 찾아 **0건**.
- 이 문서와 확인 기록의 세션 값은 앞 4글자만 남기고 가렸다. 기록 문서 안 세션 값 원문 등장 **0회**.

카드 4 — 남의 자료가 안 열리는 것을 보이기

남길 것

- 계정 두 개에 자료를 넣은 기록(비밀번호 없이)
- 양방향 읽기·수정·삭제 요청과 그 거절 응답
- 주소·헤더·본문에 남의 계정을 적어 보낸 요청과 그 응답
- 로그인하지 않은 직접 요청과 그 거절 응답
- 목록 응답에 남의 자료가 없는 결과
- 거절 앞뒤로 반대편 자료가 그대로인 결과와, 그 거절을 만드는 소스 위치

4-1. 계정 두 개와 자료 (C116)

A = `chk_a_c4a1e7`, B = `chk_b_c4a1e7` (비밀번호는 무작위, 적지 않음). 각각 계획 1 · 할 일 2 · 실행 기록 1을 넣었다. A: 계획 #7, 할 일 #18·#19, 기록 #7 / B: 계획 #8, 할 일 #20·#21, 기록 #8.

4-2. A → B: 주인일 때 성공, 남일 때 거절 (C117·C118·C119·C121)

읽기 성공 — A가 자기 할 일

```
GET /api/tasks/18/logs
Cookie: __Host-pds_sid=NDOW…(가림)

→ HTTP 200
[{"id":"7","task_id":"18","started_at":"2026-10-12T01:00:00.000Z","ended_at":"2026-10-12T01:30:00.000Z","actual_minutes":30,"blocker":"","created_at": …(생략)
```

읽기 거절 — A가 B의 할 일

```
GET /api/tasks/20/logs
Cookie: __Host-pds_sid=NDOW…(가림)

→ HTTP 404
{"error":"그 할 일을 찾을 수 없습니다."}
```

수정 성공 — A가 자기 할 일

```
PUT /api/tasks/19
Content-Type: application/json
X-Requested-With: pds-diary
Cookie: __Host-pds_sid=NDOW…(가림)

{"title":"A 할 일 2","due_date":"2026-10-13","priority":2,"tags":["시험"],"estimated_minutes":30}

→ HTTP 200
{"id":"19","plan_id":"7","title":"A 할 일 2","due_date":"2026-10-13","priority":2,"tags":["시험"],"estimated_minutes":30,"created_at":"2026-10-13T10:45:02 …(생략)
```

수정 거절 — A가 B의 할 일

```
PUT /api/tasks/20
Content-Type: application/json
X-Requested-With: pds-diary
Cookie: __Host-pds_sid=NDOW…(가림)

{"title":"남이 고침","due_date":"2026-10-20","priority":1,"tags":[],"estimated_minutes":1}

→ HTTP 404
{"error":"그 할 일을 찾을 수 없습니다."}
```

삭제 성공 — A가 자기 할 일

```
DELETE /api/tasks/22
X-Requested-With: pds-diary
Cookie: __Host-pds_sid=NDOW…(가림)

→ HTTP 200
{"deleted":true}
```

삭제 거절 — A가 B의 할 일

```
DELETE /api/tasks/21
X-Requested-With: pds-diary
Cookie: __Host-pds_sid=NDOW…(가림)

→ HTTP 404
{"error":"그 할 일을 찾을 수 없습니다."}
```

A → B 나머지 거절: `GET /api/tasks?plan_id=8`, `/api/plans/8/revisions`, `/api/plans/8/review`, `PUT /api/plans/8`, `POST /api/tasks/21/complete`, `POST /api/tasks`(B 계획에 끼워 넣기), `DELETE /api/logs/8` — 모두 404. 거절은 존재 자체를 감추는 404.

4-3. 반대 방향 B → A (C120)

읽기 거절 — B가 A의 할 일

```
GET /api/tasks/18/logs
Cookie: __Host-pds_sid=Bv-F…(가림)

→ HTTP 404
{"error":"그 할 일을 찾을 수 없습니다."}
```

수정 거절 — B가 A의 할 일

```
PUT /api/tasks/18
Content-Type: application/json
X-Requested-With: pds-diary
Cookie: __Host-pds_sid=Bv-F…(가림)

{"title":"남이 고침","due_date":"2026-10-20","priority":1,"tags":[],"estimated_minutes":1}

→ HTTP 404
{"error":"그 할 일을 찾을 수 없습니다."}
```

삭제 거절 — B가 A의 할 일

```
DELETE /api/tasks/19
X-Requested-With: pds-diary
Cookie: __Host-pds_sid=Bv-F…(가림)

→ HTTP 404
{"error":"그 할 일을 찾을 수 없습니다."}
```

삭제 거절 — B가 A의 실행 기록

```
DELETE /api/logs/7
X-Requested-With: pds-diary
Cookie: __Host-pds_sid=Bv-F…(가림)

→ HTTP 404
{"error":"그 실행 기록을 찾을 수 없습니다."}
```

B가 자기 자료에 같은 읽기·수정·삭제를 보내면 200, 200, 200. B → A 나머지 6가지도 모두 404.

4-4. 거절 앞뒤 건수 (C122)

	거절 시험 전 (본인 내보내기로 셈)	거절 시험 뒤
B의 자료 (A가 건드림)	계획 1 · 계획 이력 1 · 할 일 2 · 실행 기록 1 · 완료 기록 0	같음 (제목·고친 시각까지 같고 새로 생긴 자료 없음)
A의 자료 (B가 건드림)	전후 같음 (주인 시험으로 A가 직접 만든·지운 할 일 제외)	

4-5. 주소·머리글·본문에 다른 계정을 적어 보냄 (C123)

주소·머리글에 B(사용자 10)를 적어도 → A의 계획만

```
GET /api/plans?user_id=10&login_id=chk_b_c4a1e7
X-User-Id: 10
X-Login-Id: chk_b_c4a1e7
Cookie: __Host-pds_sid=NDOW…(가림)

→ HTTP 200
[{"id":"7","current_revision":1,"title":"A의 계획","start_date":"2026-10-12","end_date":"2026-10-16","priority":2,"success_criteria":"시험용","estimated_min …(생략)
```

본문에 B를 적어도 → A의 계획으로 만들어짐, B 자료 건수 그대로

```
POST /api/plans
Content-Type: application/json
X-Requested-With: pds-diary
Cookie: __Host-pds_sid=NDOW…(가림)

{"title":"본문에 B를 적은 계획","start_date":"2026-10-12","end_date":"2026-10-16","priority":2,"success_criteria":"시험용","estimated_minutes":60,"user_id":"10", …(생략)

→ HTTP 201
{"id":"9","current_revision":1,"title":"본문에 B를 적은 계획","start_date":"2026-10-12","end_date":"2026-10-16","priority":2,"success_criteria":"시험용","estimat …(생략)
```

4-6. 로그인하지 않은 직접 요청 (C124)

거절

```
GET /api/tasks/18/logs

→ HTTP 401
{"error":"로그인이 필요합니다."}
```

거절

```
DELETE /api/tasks/18
X-Requested-With: pds-diary

→ HTTP 401
{"error":"로그인이 필요합니다."}
```

로그인 없이 `/api/export`, `/api/home`도 401.

4-7. 목록에 남의 자료가 섞이지 않음 (C125)

A로 목록 5종(계획 목록·할 일 목록·홈·계획 요약·내보내기)을 불러 B의 ID와 겹치는 것을 셌다 → **0개**.

A의 응답에 나온 ID	계획 목록 [7, 9], 할 일 목록 [18, 19], 홈 [18, 19], 계획 요약 [7, 9], 내보내기 할 일 [18, 19, 22(지움)]
B의 ID	계획 [8], 할 일 [20, 21, 23], 실행 기록 [8]

A의 할 일 목록 응답 (B의 할 일 #20·#21 없음)

```
GET /api/tasks?plan_id=7
Cookie: __Host-pds_sid=NDOW…(가림)

→ HTTP 200
[{"id":"18","plan_id":"7","title":"A 할 일 1","due_date":"2026-10-13","priority":2,"tags":["시험"],"estimated_minutes":30,"created_at":"2026-10-13T10:45:0 …(생략)
```

4-8. 거절을 만드는 소스 위치 (C126)

줄 번호는 `server.js`, 커밋 `ac016f2` 기준. 로그인한 사람의 ID(`req.user.id`)는 세션에서만 얻고, 주소·머리글·본문의 user_id는 읽지 않는다. 모든 SQL의 `$1`이 이 ID다.

위치	하는 일
L287 로그인 확인 미들웨어 → L216 `currentUser()`	쿠키 해시가 `sessions`에 있고 만료 전인지. 아니면 401
L351 `ownPlan()` · L355 `ownTask()`	`WHERE id=$2 AND user_id=$1`로 주인 확인, 아니면 404
L441 `TASK_SELECT`	할 일 목록·홈·돌아보기가 함께 쓰는 조회에 `t.user_id = $1`
L467 할 일 목록	남의 계획 번호면 빈 목록이 아니라 404
L514 · L527 · L620	`UPDATE/DELETE … WHERE id=$2 AND user_id=$1` — 남의 것이면 한 줄도 바뀌지 않고(rowCount 0) 404
L402 계획 고치기	주인 확인(`FOR UPDATE`)을 먼저 하고, 아니면 이력도 쓰지 않음

카드 5 — 설명서로 묶고, 잠근 앱으로 5일 써 보기

남길 것

- 여섯 항목으로 나눈 인증 구현 설명서
- 확인 다섯 가지의 성공 요청과 거절 요청
- 서로 다른 날짜 5일의 기록과, 2일차 뒤 3일차 앞의 규칙 변경
- 화면 합계·평균과 손으로 더한 값의 대조
- 내보낸 파일 한 개와 계정 삭제 안내
- 비밀번호·토큰·비밀키가 모두 가려진 제출물

5-1. 인증 구현 설명서 여섯 항목 (C127~C131)

`SUBMISSION.md`에 ① 무엇으로 붙였나 · ② 왜 그걸 골랐나 · ③ 어디를 어떻게 고쳤나(가입·로그인·로그아웃·자료 조회 네 흐름의 소스 위치) · ④ 안 열리는 것을 확인한 기록 · ⑤ AI와 나 · ⑥ 아직 못 막은 것(9가지와 위험)으로 나누어 적었다. 확인 다섯 가지의 성공·거절 요청은 이 문서의 카드 1·3·4에 나란히 있다.

⑥ 아직 못 막은 것 (일부)	왜 위험한가
무차별 대입은 일부만 막음(같은 아이디 15분 5회 → 429, IP 단위 제한 없음)	쉬운 비밀번호 계정은 뚫릴 수 있고, 남이 내 아이디로 일부러 틀려 내 로그인을 막을 수 있다
비밀번호 재설정·2단계 인증 없음	잊으면 기록을 못 되찾고, 비밀번호가 새면 그것만으로 들어온다
훔친 쿠키는 최대 7일 통함, 로그인 기록을 볼 수 없음	몰래 들어와도 알아차리기 어렵다

5-2. 1일차에 고정한 것 (C04~C06·C08·C23~C27)

질문	하루에 마감으로 잡는 할 일 개수를 줄이면, 그날 계획한 할 일을 끝내는 비율이 올라갈까?
지표 · 단위	하루 계획 완료율 · %
계산 규칙	그날(서울 날짜) 마감 할 일 중 밤에 기록할 때 완료로 체크된 수 ÷ 그날 마감 할 일 수 × 100 (지운 할 일 제외)
빠짐 / 중복 / 튐	"값 없음"으로 두고 합계·평균에서 뺌 / 하루 한 번만, 두 번째 저장은 거절 / 0~100 밖은 저장 안 함
반올림 · 주 시작	소수 첫째 자리(둘째 자리에서 반올림) · 월요일
고정 시각	2026-10-13 19:03 (DB 트리거로 고칠 수 없음)

5-3. 5일 기록과 규칙 변경 (C07·C09~C12)

순서	날짜(서울)	마감 할 일	값	계획 규칙	저장 시각
1일차	2026-10-13 (화)	5개 중 4개	80.0 %	1판	10-13 23:34
2일차	2026-10-14 (수)	3개 중 3개	100.0 %	1판	10-14 23:41
규칙 변경	—	—	—	**1판 → 2판**	**10-15 23:46**
3일차	2026-10-16 (금)	4개 중 4개	100.0 %	2판	10-16 22:54
4일차	2026-10-17 (토)	3개 중 3개	100.0 %	2판	10-17 23:58
5일차	2026-10-18 (일)	3개 중 3개	100.0 %	2판	10-18 23:50

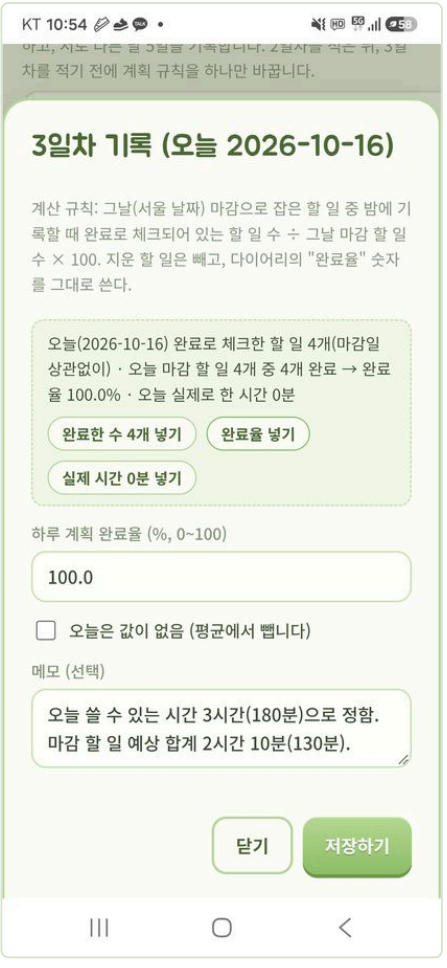
바꾼 규칙 (변경 #1, 근거: 기록 #1·#2): "그날 하고 싶은 일을 개수 제한 없이 모두 그날 마감 할 일로 붙인다" → "아침에 오늘 쓸 수 있는 시간을 먼저 정하고, 마감 할 일의 예상 시간 합계가 그 안에 들어가게 붙인다. 넘치는 일은 다음 날 마감으로 붙인다."

바꾼 이유: 1일차에 하고 싶은 일을 모두 마감으로 잡았더니 5개 중 1개를 시간 부족으로 못 끝냈다(80.0%). 2일차는 3개만 잡아 모두 끝냈다(100.0%). 개수를 정해 두기보다 그날 쓸 수 있는 시간에 맞춰 잡는 게 실제 하루와 맞을 것 같다.

날짜는 서버가 저장 순간의 서울 날짜로 정한다. 규칙 변경은 기록이 정확히 2개일 때만 한 번 저장되고 1·2일차 기록 번호를 함께 저장한다. 10/15는 규칙만 바꾸고 기록하지 않았다(밤늦게 바꿔 그날 할 일을 새 규칙으로 계획할 수 없었음).



다이어리 5일 실험 화면 (2일차 저장 직후): 1일차 기록 #1 80.0%, 2일차 기록 #2 100.0%, 그 다음 "규칙 변경" 자리



3일차 기록 창: 완료율 넣기 → 100.0, 메모에 쓸 수 있는 시간

5-4. 바꾸기 전과 후 — 화면 합계·평균과 손으로 더한 값 (C13~C15·C132)

같은 지표(하루 계획 완료율) · 같은 단위(%) · 같은 계산 규칙.

구간	센 날	화면 합계	화면 평균	손으로 더한 식	일치
전 (1·2일차)	2	180.0	90.0 %	80.0 + 100.0 = 180.0, 180.0 ÷ 2 = 90.0	✓
후 (3~5일차)	3	300.0	100.0 %	100.0 + 100.0 + 100.0 = 300.0, 300.0 ÷ 3 = 100.0	✓
5일 전체	5	480.0	96.0 %	80.0 + 100.0 × 4 = 480.0, 480.0 ÷ 5 = 96.0	✓

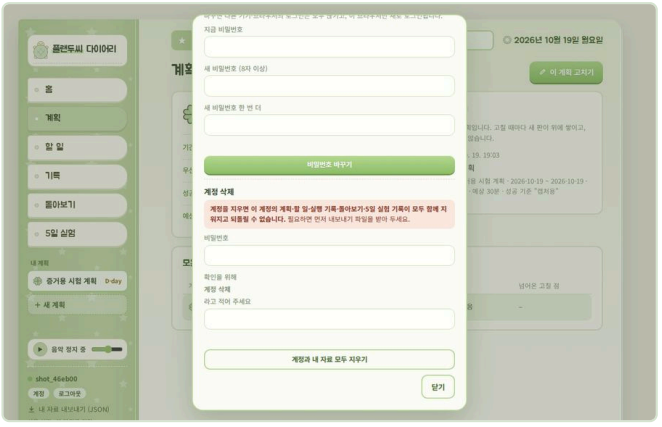
후 평균 − 전 평균 = 100.0 − 90.0 = **+10.0 %p**. 마감 할 일 수는 전 평균 4.0개 → 후 평균 3.3개. 다만 2일차는 규칙을 바꾸기 전인데도 이미 100%여서, 오른 10%p는 1일차 하루의 차이에서 나왔고 날 수도 적어 규칙 효과라고 단정하기는 어렵다.

원본: 다이어리 "제출용 기록 받기 (.md)" → `docs/evidence/5day-experiment-2026-10-19.md`

5-5. 내보낸 파일 한 개 · 계정 삭제 안내 (C133·C134)



계정 창: "내 자료 전체 내보내기" — 파일 하나(JSON), 비밀번호·로그인 정보는 들어가지 않음



계정 삭제 안내: "계획·할 일·실행 기록·돌아보기·5일 실험 기록이 모두 함께 지워지고 되돌릴 수 없습니다"

```
내보낸 파일 (시험 계정 shot_46eb00, 앞부분)
{ "format": "pds-diary-export", "version": 3, "exported_at": "2026-10-19T10:03:48.544Z",
  "account": { "login_id": "shot_46eb00", ... },
  "plans": [ { "id": "11", "title": "증거용 시험 계획", ... } ],
  "plan_revisions": [...], "tasks": [...], "task_completions": [...], "work_logs": [...],
  "reviews": [...], "experiment": ..., "experiment_days": [...], "experiment_rule_change": ... }

계정 삭제  DELETE /api/account  → HTTP 200
{"deleted":true,"removed":{"plans":1,"tasks":0,"work_logs":0,"reviews":0,"experiment_days":0}}
삭제 뒤 같은 쿠키 → 401, 같은 아이디·비밀번호로 로그인 → 401
```

5-6. 비밀번호·토큰·비밀키가 모두 가려진 제출물 (C46·C131)

점검	결과
확인 기록 응답 70여 개의 본문·머리글 전체에서 비밀번호 원문	0회
확인 기록 문서 안 세션 값 원문	0회 (앞 4글자만)
`git log -p --all`에서 DB 접속 주소(비밀번호 포함 형태)·Supabase 비밀키·JWT 모양	0건
배포되는 화면 파일 `public/`에서 접속 주소·키	0건
캡처에 쓴 시험 계정 비밀번호	무작위로 만들고 출력하지 않음, 결과 파일 전체에서 0회 확인
진짜 계정(`happyeuni`) 비밀번호	어디에도 적지 않음

휴대폰 화면



휴대폰 크기(390px)에서 연 첫 화면 — 로그인 화면